Accelerating Gradient Boosting Decision Trees via Hardware Ray Tracing Cores and On-Device Fused Reduction

GAFIME Project
Technical Disclosure Draft

July 2026

Abstract

Tree ensembles and tree-derived decision-path features are among the most effective tools for tabular learning, but large-scale candidate scoring remains limited by candidate-row predicate evaluation, intermediate membership materialization, and reduction of binary indicators against a target. This paper describes an implementation that maps bounded, shallow gradient-boosted decision-tree (GBDT) path regions onto NVIDIA hardware ray tracing cores using OptiX. A decision path over one to three numerical features is represented as an axis-aligned region, a tabular row is represented as a point, and path membership becomes a point-in-region query. The system keeps Python and Rust in charge of public API behavior, validation, scheduling, and result ownership, while CUDA owns the RT-specific C ABI, finite box validation, acceleration structure construction, ray generation, exact hit filtering, and compact score production.

The main performance contribution is not faster materialization of a dense row-by-path membership matrix. Instead, traversal results are reduced on device into compact per-path statistics and exported as ordinary Gafime result rows. For tree-leaf-like batches whose regions are finite, bounded, two-dimensional, and non-overlapping within each group, a first-hit traversal mode terminates after the first exact in-region hit and fails closed if the non-overlap proof is unavailable. On an RTX 4060 Laptop GPU, the parity-covered first-hit benchmark evaluates 262,144 $\times$ 8,192 membership-equivalent tests in 0.877–0.942 ms, or approximately 2.28–2.45 trillion evaluations/s, with maximum absolute score difference 1.29454×10^{-7} against the CPU reference. A larger throughput-only run evaluates 262,144 $\times$ 1,048,576 membership-equivalent tests in 20.030 ms, or 13.724 trillion evaluations/s. These results do not claim that ray tracing cores accelerate all GBDT training or inference. They identify a narrower systems primitive: bounded decision-path scoring can be mapped to fixed-function traversal and fused reduction when candidate geometry is tree-like and output remains compact.

Keywords: GBDT systems; RT cores; OptiX; candidate scoring; fused reduction; tabular learning.

1 Introduction

Gradient boosting decision trees are a standard workhorse for tabular machine learning. Friedman formulated gradient boosting as greedy function approximation [1]; XGBoost [2], LightGBM [3], and CatBoost [4] showed that practical performance depends as much on systems decisions as on statistical modeling. Cache-aware layout, sparsity handling, histogram construction, categorical encoding, and parallelism choices decide whether a tree system scales.

This work targets a related but distinct problem: scoring very large numbers of tree-derived decision-path candidates as explicit features. In Gafime, a candidate is not a materialized vector by default. It is a compact expression or path descriptor that is evaluated, scored, ranked, and only

materialized when an export path explicitly asks for it. The scale problem is therefore not ordinary matrix multiplication. It is candidate-row work: many candidate equations must be evaluated over many rows while retaining bounded top-k state and compact metric rows.

A shallow decision path is a conjunction of threshold predicates. For example,

$$x_{f_0} > a \quad \wedge \quad x_{f_1} \leq b.$$

For one, two, or three participating features, this conjunction is naturally an interval, rectangle, or box. Hardware ray tracing units are specialized to traverse spatial acceleration structures and answer geometric intersection queries at high throughput. NVIDIA OptiX provides a programmable ray tracing system for GPUs [10], and RTX-class GPUs include dedicated ray traversal and intersection hardware [11]. This suggests the following mapping:

$$\text{row} \longrightarrow \text{point}, \qquad \text{decision path} \longrightarrow \text{bounded region}.$$

The mapping is useful only if three engineering constraints hold. First, the geometric result must preserve the original predicate semantics, including strict and non-strict inequalities. Second, the output must remain compact; a fast path that writes a dense membership matrix moves the bottleneck rather than removing it. Third, backend ownership must remain explicit. Rust owns candidate semantics and scheduling; CUDA may execute a validated request but must not discover candidates, change fallback policy, or alter public Python behavior.

This paper is a technical disclosure of the Gafime v1 CUDA RT-core implementation. It makes the following contributions:

1. It formalizes a geometric mapping for finite, bounded, low-dimensional GBDT path regions.
2. It describes a CUDA C ABI that scores decision paths through OptiX and exports compact result rows instead of dense path-major membership.
3. It introduces a fail-closed first-hit mode for non-overlapping tree-leaf-like regions, together with the equivalence condition that makes first-hit traversal semantically valid.
4. It reports measured performance on a laptop Ada GPU, including parity-covered first-hit results and throughput-only scale runs, while separating correctness evidence from raw throughput evidence.

2 Background and Related Work

2.1 GBDT Systems

XGBoost combines gradient tree boosting with a sparsity-aware split finder, weighted quantile sketching, and cache-conscious execution [2]. LightGBM improves histogram-based training through gradient-based one-side sampling and exclusive feature bundling [3]. CatBoost addresses target leakage and categorical features through ordered boosting and specialized categorical handling [4]. GPU-oriented tree systems such as ThunderGBM [5] and GPU histogram training work [6] show that tree workloads can benefit from GPU parallelism when memory access, histogram conflicts, and algorithm structure are matched to the device.

The work here is not a replacement for these training systems. It focuses on candidate scoring for decision-path-derived features. That distinction matters: training chooses splits and updates an ensemble, while Gafime evaluates a large candidate set and keeps compact scores. The relevant unit of work is a membership-equivalent candidate-row evaluation rather than a tree-training iteration.

2.2 Tree Inference and Compilers

Tree deployment systems such as Treelite [7] and optimized forest inference paths such as NVIDIA FIL [8] target fast prediction for already-trained forests. Compiler approaches for tree inference, including Treebeard [9], lower tree traversal into optimized code for a selected target. These systems optimize inference of model outputs. GAFIME instead scores candidate path features against a target and therefore needs compact reductions over path membership rather than final ensemble prediction alone.

2.3 Ray Tracing Systems

OPTIX is a general-purpose programmable ray tracing engine [10]. Dedicated RTX hardware exposes fixed-function traversal and intersection acceleration [11]. GPU ray traversal work has studied how traversal efficiency depends on scheduling, memory locality, divergence, and scene structure [12, 13]. Embree [14] demonstrated that carefully engineered ray traversal kernels can be useful reusable systems infrastructure on CPUs. This paper applies ray traversal to a tabular candidate-scoring problem: not rendering, but evaluating whether row-points belong to many decision-path regions.

3 Decision Paths as Regions

Let $X \in \mathbb{R}^{n \times d}$ be a tabular matrix with n rows and d features, and let $y \in \mathbb{R}^n$ be a target. A path candidate p is a conjunction of threshold terms:

$$p = \{(f_\ell, s_\ell, t_\ell)\}_{\ell=1}^{L},$$

where f_ℓ is a feature id, $s_\ell \in \{\leq, >\}$ is the comparison, and t_ℓ is a threshold. Membership for row i is

$$z_{p,i} = \begin{cases} 1, & \text{if all terms in } p \text{ hold for row } i, \\ 0, & \text{otherwise.} \end{cases}$$

Definition 1 (Representable RT path)**.** *A path batch is representable by the RT backend when every path in the batch has finite thresholds, all referenced feature values are finite, and each internal geometry group uses at most three feature axes. Bounded two-dimensional groups may use triangle geometry; one-dimensional, three-dimensional, or open-bound groups require exact custom-AABB handling when supported.*

Each term tightens a lower or upper bound along a feature axis. For a path over unique feature axes $(u_1, \ldots, u_k)$, $k \leq 3$, the conjunction maps to

$$B_p = I_{p,1} \times \cdots \times I_{p,k},$$

where $I_{p,j}$ may be lower-open or upper-closed depending on the original predicate. A row maps to

$$q_i = (X_{i,u_1}, \ldots, X_{i,u_k}).$$

The desired semantic test is $z_{p,i} = 1$ if and only if $q_i \in B_p$ under the same open/closed comparison rules as the original path.

$$\begin{aligned} \text{GBDT path} &: x_{f_0} > a \ \wedge \ x_{f_1} \le b \\ \text{Region} &: (a, \infty) \times (-\infty, b] \\ \text{Row} &: q_i = (X_{i,f_0}, X_{i,f_1}) \\ \text{Score} &: \rho(z_p, y) \text{ or } R^2(z_p, y) \end{aligned}$$

candidate descriptors → RT traversal → on-device counts/sums → compact result rows.

Figure 1: Decision-path scoring as point-in-region traversal followed by compact on-device reduction. The implementation does not export a dense row-by-path matrix in the normal scoring path.

4 Scoring Without Dense Membership

The compact score path currently supports Pearson correlation and R^2 for decision-path indicators. For path p, define

$$S_x = \sum_i z_{p,i}, \qquad S_y = \sum_i y_i, \qquad S_{yy} = \sum_i y_i^2, \qquad S_{xy} = \sum_i z_{p,i} y_i.$$

The centered terms are

$$C_{xx} = S_x - \frac{S_x^2}{n}, \qquad C_{yy} = S_{yy} - \frac{S_y^2}{n}, \qquad C_{xy} = S_{xy} - \frac{S_x S_y}{n}.$$

The Pearson score is

$$\rho_p = \begin{cases} \operatorname{clip}\left(C_{xy}/\sqrt{C_{xx}C_{yy}}, -1, 1\right), & C_{xx}C_{yy} > 0, \\ 0, & \text{otherwise}, \end{cases}$$

and $R_p^2 = \operatorname{clip}(\rho_p^2, 0, 1)$.

This formula shows why dense membership export is unnecessary. The score needs only per-path counts and target sums plus target-wide statistics. Therefore, the backend can evaluate membership transiently, reduce on device, and return one compact metric row per candidate.

5 System Design

5.1 Ownership Boundary

GAFIME v1 uses a hard ownership split. Python exposes the public API; Rust owns validation, feature planning, scheduling, backend selection, and report ownership; CUDA owns the CUDA-specific execution boundary. The RT path therefore has the following contract:

- Rust passes compact, validated decision-path descriptors through the GPU C ABI.
- CUDA validates RT representability, builds geometry, launches OPTIX, and writes compact result rows.
- CUDA does not discover candidate paths, mutate scheduler policy, infer Python behavior, or silently fall back to another backend.
- Unsupported RT requests return unsupported when RT is required or when first-hit mode cannot be proven safe.

5.2 CUDA Source Split

The implementation keeps RT code out of the generic continuous CUDA kernels:

- `src/cuda/rt_kernels.cu` contains OptiX device programs, RT-specific CUDA kernels, point packing, exact hit filters, and direct-score accumulation helpers.
- `src/cuda/rt_launcher.cu` owns host-side RT planning, finite-box validation, OptiX pipeline setup, GAS/IAS construction, cached workspaces, grouped execution, and explicit comparator dispatch.
- `src/cuda/launcher.cu` remains the public C ABI bridge for decision-path RT symbols.

This source layout is part of the architectural contract. RT traversal logic is large enough to deserve explicit ownership; hiding it inside the generic CUDA score path would make later backend maintenance harder.

5.3 Grouped Instanced Launch

Candidate batches may contain many feature pairs. A single triangle GAS cannot interpret point coordinates for all feature-pair coordinate systems at once. The CUDA launcher groups paths by compatible feature axes, builds one triangle GAS per group, and wraps group GAS handles in one instance acceleration structure (IAS). Ray generation then launches

$$\text{launch shape} = (n,\ g,\ 1),$$

where g is the number of RT groups. The instance id selects the group-local path table, and a scatter map restores original public path order before result export. This keeps Rust in charge of scheduling while giving OptiX a large batched launch.

5.4 Score Modes

Bitset mode. OptiX writes an idempotent device bitset, and a CUDA kernel reduces the bitset with the target vector. This is the conservative parity path because it avoids floating accumulation order in the traversal program.

Direct mode. The any-hit program updates per-path counts and target sums directly. This removes the temporary bitset and reduction scan but uses floating atomics for some statistics, so it is opt-in and documented with tolerance.

First-hit mode. For finite, bounded, two-dimensional groups where CUDA proves that boxes do not overlap, the any-hit program accepts the first exact hit and terminates traversal. This matches tree-leaf assignment: a row belongs to at most one leaf region in a partition.

Proposition 1 (First-hit equivalence)**.** *Let $\mathcal{B} = \{B_1, \ldots, B_m\}$ be a group of bounded two-dimensional decision regions such that the interiors are pairwise disjoint and the exact open/closed predicate guard assigns every boundary point to at most one region. For any row point q, first-hit traversal with the exact guard returns the same membership vector as evaluating all path predicates and selecting the unique inside region, if one exists.*

Proof. If q is outside all regions, no exact guard accepts a hit, so all membership values are zero. If q is inside one region B_j, disjointness and the boundary ownership guard imply no other region can also accept q. Terminating after the first accepted hit therefore records B_j and cannot suppress another valid region. The recorded membership is identical to exhaustive predicate evaluation. □

First-hit mode is fail-closed. If the finite, bounded, 2D, non-overlapping proof is unavailable, the RT score ABI returns unsupported rather than changing semantics or silently routing the opt-in request to the SM comparator.

6 Numerical Policy

The correctness policy separates Boolean membership from floating score accumulation. Membership semantics are exact with respect to the GAFIME predicate model: `LE` means $x \leq t$, `GT` means $x > t$, and NaNs are not silently reinterpreted. Uploaded matrices must satisfy the finite RT precondition or the RT path is unsupported.

Integer counts, path ids, and result row ordering are exact. Floating Pearson and R^2 values may differ when a mode relies on floating atomics or backend-specific instruction ordering. Such modes are opt-in and must report an approved tolerance. The present first-hit parity run has maximum absolute score difference 1.29454×10^{-7} against the CPU reference, well below the documented spike tolerance of 10^{-4}.

7 Evaluation

7.1 Hardware and Workloads

The measurements in this disclosure were collected on a laptop system with an AMD Ryzen AI 9 HX 370 class CPU and an NVIDIA GeForce RTX 4060 Laptop GPU (Ada, sm_89, approximately 7.63 GiB reported global memory). The benchmark is `tests/gpu/cuda_rt_membership_scale_bench.cpp`. The release-measure wrapper is `tests/release_measure/perf_05_cuda_rt_firsthit_scale.py`.

The synthetic workloads use bounded 2D boxes over feature-pair groups. The partitioned-grid mode creates non-overlapping boxes within each group, modeling the tree-leaf invariant required by first-hit traversal. Throughput is reported as membership-equivalent evaluations per second:

$$\text{evals} = n \times m,$$

where n is row count and m is path count.

7.2 Dense Membership Is the Wrong Output

Table 1 shows that materializing a dense membership matrix does not expose the RT advantage. The RT path and SM comparator are in the same low-G evaluations/s band because output traffic dominates.

Table 1: Path-major membership materialization on 1,048,576 × 512 candidate-row evaluations. The output is a dense 2.00 GiB membership-equivalent matrix.

Mode	Rows	Paths	Time (ms)	G eval/s
CPU AVX512 membership	1,048,576	512	111.218	4.827
CUDA RT membership	1,048,576	512	194.236	2.764
CUDA SM membership	1,048,576	512	189.348	2.835

7.3 Compact Scoring

Table 2 removes the dense output and reduces scores on device. The public output is one row per path per metric, not one row per path-row membership value.

Table 2: Compact score ABI on the same 1,048,576 × 512 logical candidate-row workload.

Mode	Rows	Paths	Time (ms)	G eval/s
CUDA RT compact score	1,048,576	512	9.118	58.881
CUDA SM compact score	1,048,576	512	26.768	20.056

The temporary score mask for this case is 64 MiB as a bitset, compared with 512 MiB as a byte mask and 2.00 GiB as an `f32` membership matrix.

7.4 Grouped Direct Score

Direct traversal statistics remove the bitset and are useful for studying RT saturation, but they are not the default deterministic path because floating atomic ordering can shift the final score by a few units in 10^{-6}.

Table 3: Representative direct-score runs. These are performance evidence for the RT saturation direction, not the default release scoring mode.

Rows	Paths	Time (ms)	RT G eval/s	SM G eval/s	RT max abs
1,048,576	512	3.195	168.048	25.471	5.04×10^{-6}
262,144	512	1.572	85.397	20.977	7.58×10^{-6}
262,144	8,192	12.445	172.555	20.028	4.10×10^{-6}

7.5 First-Hit Partitioned Score

The strongest result appears when the regions satisfy the tree-leaf-like non-overlap condition. Table 4 gives both parity-covered and throughput-only cases.

Table 4: First-hit partitioned scoring. Throughput-only rows are not correctness evidence and must be interpreted together with parity-covered runs.

Case	Rows	Paths	Time (ms)	G eval/s	Parity
Parity-covered	262,144	8,192	0.877–0.942	2,278–2,450	1.29454×10^{-7}
Throughput-only	65,536	1,048,576	18.059	3,805	skipped
Throughput-only	262,144	1,048,576	20.030	13,724	skipped

The release-measure run for the parity-covered case reported:

resident upload	3.832 ms,
CPU score reference	6170.278 ms (0.348 G eval/s),
GPU RT score	0.942 ms (2278.804 G eval/s),
maximum absolute difference	1.29454×10^{-7}.

7.6 Interpretation

The first-hit result is large because the workload matches the traversal hardware:

- path candidates are bounded regions, not arbitrary branch programs;
- the partitioned-grid invariant lets each row-group ray terminate after the first exact hit;
- grouped instancing creates a large launch rather than many tiny launches;
- score output is compact and does not copy dense membership to host;
- resident sessions reuse geometry, packed points, target statistics, and scratch buffers when inputs are unchanged.

Dense overlapping boxes do not show the same behavior. Million-path overlap stress cases remain around 95–99 G eval/s and skip parity at that size. That result is useful because it marks the boundary of the method: RT traversal is most effective here when the candidate regions resemble tree partitions, not a dense random overlap field.

8 Limitations

The implementation is intentionally narrow. The fastest first-hit path currently applies only to finite, bounded, non-overlapping 2D groups. The compact RT decision-path score ABI supports Pearson and R^2 in this spike; MI and Spearman require additional parity work. CUDA/OptiX is the only RT backend. ROCm and Metal do not expose this RT-core path through the current Gafime ABI.

The 13.724 T eval/s number is throughput-only. It is important performance evidence for the same code path and workload family, but it is not itself a CPU-parity validation run. Correctness is established by the smaller parity-covered cases.

The evaluation uses one Ada laptop GPU. Larger desktop RTX devices or different generations may shift the bottleneck toward acceleration-structure size, target-stat atomic pressure, memory bandwidth, or launch scheduling. Available NCU counters did not expose a direct RT-core saturation percentage for the OptiX launch; visible counters suggested that ordinary branch divergence was not the primary limiter, but a deeper architecture sweep is future work.

9 Future Work

Several pieces remain before this becomes a broader GBDT acceleration method:

1. evaluate paths extracted from real trained ensembles, not only synthetic partition grids;
2. add deterministic buffered first-hit statistics and compare them with direct floating atomics;
3. extend compact RT scoring to MI and Spearman after parity is proven;
4. test Turing, Ampere, Ada, Hopper, and Blackwell devices;
5. capture repeated resident score calls with CUDA graphs where geometry and buffer shape are stable;
6. compare against production tree-inference and GPU-GBDT systems on end-to-end tasks while preserving the distinction between candidate scoring, tree inference, and training.

10 Conclusion

This paper reports a systems mapping from bounded decision-path scoring to hardware ray tracing traversal and on-device fused reduction. The key insight is not that every tree operation is geometric. The useful subset is narrower: finite low-dimensional path regions can be scored as point-in-region tests, and tree-leaf-like non-overlap makes first-hit traversal semantically valid.

In Gafime, the result is integrated without moving public semantics into the backend. Rust owns candidate planning and scheduling, CUDA owns validated RT execution, and unsupported requests fail explicitly. On a parity-covered RTX 4060 Laptop workload, the implementation reaches approximately 2.28–2.45 T membership-equivalent evaluations/s with maximum absolute score error 1.29454×10^{-7}. On a larger throughput-only workload it reaches 13.724 T evaluations/s. These results identify RT cores as a serious candidate for accelerating a specific class of decision-path feature scoring: bounded, compact, tree-like, and reducible on device.

References

[1] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. https://doi.org/10.1214/aos/1013203451.

[2] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. https://doi.org/10.1145/2939672.2939785.

[3] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, 30, 2017. https://papers.nips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree.

[4] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, 31, 2018. https://papers.nips.cc/paper/7898-catboost-unbiased-boosting-with-categorical-features.

[5] Zeyi Wen, Jiashuai Shi, Bingsheng He, Jian Chen, Kotagiri Ramamohanarao, and Qinbin Li. ThunderGBM: Fast GBDTs and random forests on GPUs. *Journal of Machine Learning Research*, 21(108):1–5, 2020. https://www.jmlr.org/papers/v21/19-095.html.

[6] Huan Zhang, Si Si, and Cho-Jui Hsieh. GPU-acceleration for large-scale tree boosting. arXiv:1706.08359, 2017. https://arxiv.org/abs/1706.08359.

[7] Hyunsu Cho and contributors. Treelite: Toolbox for decision tree deployment. Amazon Science technical report and project documentation. https://www.amazon.science/publications/treelite-toolbox-for-decision-tree-deployment.

[8] NVIDIA RAPIDS. Forest Inference Library documentation. https://docs.rapids.ai/api/cuml/stable/fil/.

[9] Brian Van Essen, Charith Mendis, Saman Amarasinghe, and collaborators. An optimizing compiler for decision tree based ML inference. In *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture*, 2022. https://doi.org/10.1109/MICRO56248.2022.00043.

[10] Steven G. Parker, James Bigler, Andreas Dietrich, Heiko Friedrich, Jared Hoberock, David Luebke, David McAllister, Morgan McGuire, Keith Morley, Austin Robison, and Martin Stich. OptiX: A general purpose ray tracing engine. *ACM Transactions on Graphics*, 29(4), 2010. https://doi.org/10.1145/1778765.1778803.

[11] NVIDIA. NVIDIA Turing GPU Architecture. White paper, 2018. https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf.

[12] Timo Aila and Samuli Laine. Understanding the efficiency of ray traversal on GPUs. In *Proceedings of High Performance Graphics*, 2009. https://users.aalto.fi/~ailat1/HPG2009/.

[13] Timo Aila, Samuli Laine, and Tero Karras. Understanding the efficiency of ray traversal on GPUs: Kepler and Fermi addendum. NVIDIA Technical Report NVR-2012-02, 2012. https://research.nvidia.com/publication/2012-06_understanding-efficiency-ray-traversal-gpus-kepler-and-fermi-addendum.

[14] Ingo Wald, Sven Woop, Carsten Benthin, Gregory S. Johnson, and Manfred Ernst. Embree: A kernel framework for efficient CPU ray tracing. *ACM Transactions on Graphics*, 33(4), 2014. https://doi.org/10.1145/2601097.2601199.